ХАРКІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ВНУТРІШНІХ СПРАВ

Кафедра інформаційних систем та технологій

ЛАБОРАТОРНА РОБОТА №10

з дисципліни «Об’єктно-орієнтовне програмування»

Тема: «Патерни проєктування для Python (Factory, Singleton, Strategy)»

Виконав:

курсант 2-го курсу

ННІ 4-25-203 інф

Кожевніков К.О.

Прийняв:

Доцент кафедри ІСТ, к.т.н.,
Слободянюк О.В.

КАМ’ЯНЕЦЬ-ПОДІЛЬСЬКИЙ – 2026

ВАРІАНТ 6. Медичний центр

Завдання:

Класи для роботи: Service, Appointment, Client. Створіть фабрику записів Appointment і дві стратегії нагадування. Singleton обговоріть лише на вигаданій демонстраційній конфігурації, не зберігайте реальних медичних даних.

КОД РОЗВ'ЯЗКУ

```
from abc import ABC, abstractmethod

# Словник для реєстрації типів записів за допомогою декоратора класу
APPOINTMENT_TYPES = {}

def register_appointment_type(cls):
    """Декоратор класу: реєструє тип запису за його унікальним атрибутом kind."""
    if hasattr(cls, "kind"):
        APPOINTMENT_TYPES[cls.kind] = cls
    return cls

class AuditableMixin:
    """Mixin-клас, що додає функціонал логування (аудиту) подій для запису."""

    def audit_record(self):
        """Повертає рядок для запису в журнал подій медичного центру."""
        return f"[АУДИТ] Запис №{self.number} (час: {self.time}): змінено стан або створено подію."

class Service:
    """Модель медичної послуги."""

    def __init__(self, name, duration_min):
        self.name = name
        self.duration_min = duration_min

    def __str__(self):
        return f"Послуга «{self.name}» ({self.duration_min} хв)"

    def __repr__(self):
        return f"Service(name={self.name!r}, duration_min={self.duration_min!r})"
```

```
        def __eq__(self, other):
            if not isinstance(other, Service):
                return NotImplemented
            return self.name == other.name and self.duration_min ==
other.duration_min

    class Client:
        """Модель пацієнта (клієнта) медичного центру."""

        def __init__(self, full_name, phone):
            self.full_name = full_name
            self.phone = phone

        def __str__(self):
            return f"{self.full_name} (тел: {self.phone})"

        def __repr__(self):
            return f"Client(full_name={self.full_name!r},
phone={self.phone!r})"

    @register_appointment_type
    class Appointment(AuditableMixin, ABC):
        """Абстрактний базовий клас запису на прийом."""

        kind = "base"
        valid_statuses = {"підтверджено", "завершено", "скасовано"}
        created_count = 0

        def __init__(self, number, time, service, client,
status="підтверджено"):
            self.number = number
            self.time = time
            self.service = service
            self.client = client

            self._status = "підтверджено"
            self.status = status

            Appointment.created_count += 1

        @property
        def status(self):
            """Повертає поточний статус із захищеного внутрішнього
поля."""
            return self._status

        @status.setter
        def status(self, value):
```

```
            """Дозволяє встановити лише допустимий статус."""
            if value not in self.valid_statuses:
                raise ValueError(f"Невідомий статус: {value}")
            self._status = value

        @staticmethod
        def is_valid_number(number):
            """Статичний метод: перевіряє, чи є номер додатним
цілим числом."""
            return isinstance(number, int) and number > 0

        @classmethod
        def from_dict(cls, data, service, client, **kwargs):
            """Альтернативний конструктор з урахуванням специфічних
полів."""
            return cls(
                number=data["number"],
                time=data["time"],
                service=service,
                client=client,
                status=data.get("status", "підтверджено"),
                **kwargs
            )

        @abstractmethod
        def preparation_note(self):
            """Абстрактний метод підготовки до прийому."""
            pass

        def __str__(self):
            return (
                f"Запис №{self.number} | Час: {self.time} | "
                f"Пацієнт: {self.client.full_name} | Послуга:
{self.service.name} | Статус: {self.status}"
            )

        def __repr__(self):
            return (
                f"{self.__class__.__name__}(number={self.number!r},
time={self.time!r}, "
                f"service={self.service!r}, client={self.client!r},
status={self.status!r})"
            )

        def __eq__(self, other):
            if not isinstance(other, Appointment):
                return NotImplemented
            return self.number == other.number
```

```
    @register_appointment_type
    class Consultation(Appointment):
        """Похідний клас для консультаційних прийомів лікаря."""

        kind = "consultation"

        def __init__(self, number, time, service, client,
doctor_office="101", status="підтверджено"):
            super().__init__(number, time, service, client, status)
            self.doctor_office = doctor_office

        @classmethod
        def from_dict(cls, data, service, client):
            return cls(
                number=data["number"],
                time=data["time"],
                service=service,
                client=client,
                doctor_office=data.get("doctor_office", "101"),
                status=data.get("status", "підтверджено")
            )

        def preparation_note(self):
            return f"Консультація у каб. №{self.doctor_office}:
очікуйте під кабінетом, прийдіть за 10 хв."

    @register_appointment_type
    class DiagnosticVisit(Appointment):
        """Похідний клас для діагностичних процедур."""

        kind = "diagnostic"

        def __init__(self, number, time, service, client,
equipment_code="EQ-0000", status="підтверджено"):
            super().__init__(number, time, service, client, status)
            self.equipment_code = equipment_code

        @classmethod
        def from_dict(cls, data, service, client):
            return cls(
                number=data["number"],
                time=data["time"],
                service=service,
                client=client,
                equipment_code=data.get("equipment_code", "EQ-
0000"),
                status=data.get("status", "підтверджено")
            )
```

```
        def preparation_note(self):
            return f"Діагностика (обладнання код:
{self.equipment_code}): дотримуйтеся інструкцій лікаря."

    # --- 1. ПАТЕРН ФАБРИКА (Factory) ---
    class AppointmentFactory:
        """Фабрика для створення екземплярів записів за текстовим
ключем (kind)."""

        @classmethod
        def create_appointment(cls, kind, number, time, service,
client, **kwargs):
            """Повертає екземпляр конкретного класу запису з
реєстру."""
            appointment_cls = APPOINTMENT_TYPES.get(kind)
            if not appointment_cls:
                raise ValueError(f"Невідомий тип запису: '{kind}'")
            return appointment_cls(number, time, service, client,
**kwargs)

    # --- 2. ПАТЕРНИ СТРАТЕГІЇ НАГАДУВАННЯ (Strategy) ---
    class StandardReminderStrategy:
        """Стандартна стратегія нагадування (звичайна SMS-
розсилка)."""

        def send_reminder(self, appointment):
            return f"[SMS стандартне] Нагадування пацієнту
{appointment.client.full_name}: ваш запис №{appointment.number} на
{appointment.time} підтверджено."

    class UrgentReminderStrategy:
        """Термінова стратегія нагадування (дзвінок або термінове
повідомлення)."""

        def send_reminder(self, appointment):
            return f"[ТЕРМІНОВО / ДЗВІНОК] Увага,
{appointment.client.full_name}! Скоро ваш прийом
({appointment.time}), будь ласка, не запізнюйтеся."

    class AppointmentProcessor:
        """Контекст, який делегує виконання генерації нагадування
обраній стратегії."""

        def __init__(self, strategy):
            self.strategy = strategy

        def set_strategy(self, strategy):
            """Дозволяє динамічно змінювати стратегію під час
виконання програми."""
```

```
            self.strategy = strategy

        def process_reminder(self, appointment):
            """Викликає алгоритм нагадування відповідно до поточної
стратегії."""
            return self.strategy.send_reminder(appointment)

    # --- 3. ПАТЕРН SINGLETON (Для демонстраційної конфігурації) --
-
    class ClinicSettings:
        """Навчальний приклад Singleton для спільних налаштувань
медичного центру."""
        _instance = None

        def __new__(cls):
            if cls._instance is None:
                cls._instance = super().__new__(cls)
                # Ініціалізуємо демонстраційні налаштування лише
один раз
                cls._instance.clinic_name = "Медичний Центр
«Здоров'я»"
                cls._instance.working_hours = "08:00 - 20:00"
            return cls._instance

    # --- ДЕМОНСТРАЦІЯ РОБОТИ ---
    if __name__ == "__main__":
        print("=== 1. Перевірка патерну Фабрика (Factory) ===")
        s1 = Service("Консультація терапевта", 30)
        s2 = Service("УЗД черевної порожнини", 40)
        c1 = Client("Олександр Т.", "+380501112233")
        c2 = Client("Марія Д.", "+380672223344")

        # Створюємо записи через фабрику за допомогою ключа типу
        app_consultation = AppointmentFactory.create_appointment(
            "consultation", 101, "2026-11-07 09:00", s1, c1,
doctor_office="305"
        )
        app_diagnostic = AppointmentFactory.create_appointment(
            "diagnostic", 102, "2026-11-07 10:30", s2, c2,
equipment_code="EQ-777"
        )

        print(app_consultation)
        print(f"   -> {app_consultation.preparation_note()}")
        print(app_diagnostic)
        print(f"   -> {app_diagnostic.preparation_note()}")

        print("\n=== 2. Перевірка патерну Стратегія (Strategy)
===")
```

```
        # Ініціалізуємо процесор зі стандартною стратегією
        processor =
AppointmentProcessor(StandardReminderStrategy())

        print("Виклик зі стандартною стратегією:")
        print(processor.process_reminder(app_consultation))

        # Перемикаємо стратегію на термінову без зміни клієнтського
коду
        processor.set_strategy(UrgentReminderStrategy())
        print("Виклик після динамічного перемикання на термінову
стратегію:")
        print(processor.process_reminder(app_consultation))

        print("\n=== 3. Перевірка патерну Singleton (Конфігурація)
===")
        settings_1 = ClinicSettings()
        settings_2 = ClinicSettings()

        print(f"Назва клініки з першого об'єкта:
{settings_1.clinic_name}")
        print(f"Робочі години з другого об'єкта:
{settings_2.working_hours}")
        print(f"Чи є settings_1 та settings_2 тим самим
екземпляром? {settings_1 is settings_2}")
```

РЕЗУЛЬТАТИ РОБОТИ ПРОГРАМИ

